

EEE 443/543 FALL 2025-26

Assignment # 3

Assignment # 3 : In this project we will consider handwritten character recognition by using a multilayer structure and MNIST dataset. MNIST contains total of 70.000 handwritten characters, each has 28×28 pixel (grayscale) image. 60.000 of these are reserved as training set and the remaining 10.000 are reserved as test set. You may access the dataset through :

<http://yann.lecun.com/>

<https://github.com/mkolod/MNIST#>



1 : $28 \times 28 = 784$, and the first thing to do is to convert this image to a vector of length 784. This is called flattening and simple python, numpy codes can do that. Note that due to grayscale, each pixel has a value between 0-255. To speed up the learning process, the input vectors are usually normalized (simple way is to divide each entry by 255).

2 : For classification, we will use a 2 layer structure. Output layer should contain 10 neurons, where each neuron corresponds to a digit that input pattern represents. If the network is trained correctly, the output neuron having the maximum value should represent the digit represented by the input image. There will be a hidden layer with N neurons. Choose $N = 400, 800$. Note that input and hidden layer should also contain threshold.

3 : For nonlinear activation function, we will use two cases :

Case 1 : All activation functions are $\tanh(v) = \frac{1-\exp(-v)}{1+\exp(-v)}$

Case 2 : Hidden layer activation functions are $RELU(v) = \max\{0, v\}$ and output layer activation functions are tanh functions as given above.

4 : Preassign the output neurons to digits as follows : first output neuron should correspond to digit 1, second one to digit 2, so on. The 10th output neuron should correspond to digit 0.

Desired outputs should be chosen as follows. For digit i , only the corresponding output neuron should be 1, all others should be -1, $i = 1, 2, \dots, 10$.

5 : The error to be minimized is the classical average mean squares error E_{ave} defined as : $E(n) = 0.5 \sum_{r=1}^{10} (d_r(n) - o_r(n))^2$, $E_{ave} = \frac{1}{NP} \sum_{n=1}^{NP} E(i)$. Here, n is the pattern index, NP is the number of total patterns, $d_i(n)$ and $o_i(n)$ are the desired and actual output values for i^{th} output neuron for pattern n .

6 : Choose the learning coefficient η as $\eta = 0.01, 0.03$.

7 : For the initialization of weights use random initialization, e.g. uniform distribution in a small interval around 0, such as $[-0.01, 0.01]$, or gaussian distribution with zero mean and small variance, such as $\sigma = 0.1$.

8 : For each of the cases given in **2, 3, 6**, run the back propagation algorithm, online. Note that we have $2 \times 2 \times 2$ runs. (For statistically meaningful results, one should have as many runs as possible, use the average success rates and look at the variances, etc.). For stopping condition, you may choose an upper bound on the number of epochs, typically 50-100 epochs would suffice. You may also monitor the training set error, and if the decay becomes slower you may stop the training. If the algorithm seems not converging, you may use the hint given in the previous assignment.

9 : After the convergence, give as much relevant data as possible. Record the average error in the training set, the success rate in the training set, how many misclassified patterns you have in the training for each class, number of epochs till convergence, average CPU time per epoch and the total duration spent for the training. Also record the average error in the test set, the success rate in the test set, how many misclassified patterns you have in the test set for each class, the total time it takes to classify a pattern (this is called inference time). You must show your efforts to obtain decent success rates in test (e.g. higher than 90 %). Very poor network performance will also result in a lower grade.

10 : After the step **9**, record the configuration which gives you the best test accuracy result. For this configuration **only**, run the back propagation algorithm by using batch learning with batch size $B = 20, 200$. Note that this will require 2 extra simulations. Record the same information as requested in step **9**.

11 : Write your results in a nice report format (in pdf).

Instructions:

1. Prepare a report (including your answers/plots) to be uploaded on Moodle.
 2. The report should be typeset (no handwriting allowed except for lengthy derivations, which may be scanned and embedded into the report).
 3. Show all steps of your work clearly.
 4. Unclear presentation of results will be penalized heavily.
 5. No partial credits for unjustified answers.
 6. **Use of any toolbox or library for neural networks is prohibited.**
 7. Return all XXXXXXXXXX Python code that you wrote in a single XXXXXX.py file.
 8. Code should be commented,
 9. The code file should NOT return an error during runtime.
 10. If the code returns an error at any point, the remaining part of your code will not be evaluated (i.e., 0 points).
-